WEEK-2
ADVANCE WEB PROGRAMMING

1. Introduction to React.js

References

- Chapter 1 – *Hello World*, pp. 1–7

  React.Up.and.Running.2nd.Editio…

- Preface – *About React*, p. xi

  React.Up.and.Running.2nd.Editio…

What is React?

React is a **JavaScript library** (not a framework) for building user interfaces.
It allows developers to **build reusable components** and manage **UI state efficiently**.

Key points:

- Created by **Facebook** (now Meta)
- Uses **Virtual DOM** for performance
- Focuses on **component-based architecture**
- Enables **declarative UI** (you describe the UI, React updates it)
- Works with **HTML, CSS, and JS** combined through **JSX**

Implementation Example

hello.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hello React</title>
    <meta charset="UTF-8" />
    <script
src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  </head>
```

```
  <body>
    <div id="root"></div>
    <script>
      ReactDOM.render(
        React.createElement('h1', null, 'Hello, React World!'),
        document.getElementById('root')
      );
    </script>
  </body>
</html>
```

Explanation:
`ReactDOM.render()` replaces the `div#root` content with dynamically generated elements — no full page reload is required.

Why Use React?

- Declarative syntax (less manual DOM manipulation)
- Component reuse = faster development
- Virtual DOM boosts performance
- Works well for **Single Page Applications (SPAs)**
- Huge community and ecosystem

2. React vs Traditional Web Development

Reference

- Chapter 1 – *What Just Happened?*, pp. 3–5

  React.Up.and.Running.2nd.Editio…

Aspect	Traditional Web	React-based
DOM Updates	Manual DOM manipulation (e.g., `document.getElementById()`)	Automatic via Virtual DOM
Page Structure	Multiple HTML pages	Single HTML + dynamic components
Code Reuse	Limited	Reusable Components
Rendering	Reloads entire page	Rerenders only updated parts
Data Handling	Separate JS scripts	State-driven, unidirectional data flow

Example:
Traditional JS:

```
document.getElementById('msg').innerText = "Hello!";
```

React:

```
function App() {
  const [msg, setMsg] = React.useState("Hello!");
  return <h1>{msg}</h1>;
}
```

3. Single Page Applications (SPA)

Reference

- Chapter 8 – *Routing*, pp. 194–195

  React.Up.and.Running.2nd.Editio…

Definition:
A Single Page Application loads one main HTML file and dynamically updates content without full-page reloads.

How SPA Works

- Browser loads one HTML file (`index.html`)
- React handles navigation internally (using React Router or custom logic)
- Only components are updated → faster transitions and better UX

Example Flow:

```
/home → shows <Home />
/about → shows <About />
(no page reload)
```

Code:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";

function App() {
  return (
    <BrowserRouter>
```

```
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

4. Setting up React with Vite

Reference

- Chapter 6 – *Setting Up for App Development*, pp. 119–126

  React.Up.and.Running.2nd.Editio…

The book uses *Create React App (CRA)*, but **Vite** is the modern replacement — faster and simpler.

Steps: Installing React via Vite

1. Install Node.js (required)

Download from https://nodejs.org

2. Create React app using Vite

```
npm create vite@latest my-react-app
cd my-react-app
npm install
npm run dev
```

3. Folder Structure (similar to CRA)

```
my-react-app/
├── index.html
├── package.json
├── node_modules/
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── assets/
    └── components/
```

4. Start the development server

```
npm run dev
```

Your app will run at `http://localhost:5173/`.

5. Folder Structure in React

Reference

- Chapter 7 – *Building the App's Components*, pp. 127–128

  React.Up.and.Running.2nd.Editio…

```
src/
├── components/     → UI components
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── Excel.jsx
├── modules/        → Helper functions (e.g. clone.js)
├── App.jsx
├── App.css
└── main.jsx
```

Explanation:
Keep **components** and **modules** separate for reusability and clarity.
Follow the "Import → Work → Export" pattern.

6. Import & Export in JavaScript (ES6 Modules)

Reference

- Chapter 6 – *JavaScript: Modernized*, pp. 124–125

  React.Up.and.Running.2nd.Editio…

- Chapter 7 – *Building Components*, pp. 129–130

  React.Up.and.Running.2nd.Editio…

Default Export

Used when a module exports a single thing.

```
// Greet.jsx
export default function Greet() {
  return <h2>Hello React!</h2>;
```

```
}

// App.jsx
import Greet from './Greet';
```

Named Export

Used when multiple exports exist.

```
// utils.js
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// App.jsx
import { add } from './utils';
```

 Rule of Thumb:

- Use **default export** for one main component per file.
- Use **named export** for utility functions or constants.

7. Virtual DOM vs Real DOM

Reference

- Chapter 3 – *Conclusion and Virtual DOM Diffs*, pp. 52–53

  React.Up.and.Running.2nd.Editio…

Real DOM

- Browser directly manipulates HTML elements.
- Any update re-renders **the entire page/tree**.

Virtual DOM

- React maintains a **virtual copy** of the DOM in memory.
- When state changes → React compares Virtual DOM with Real DOM.
- Only **differences (diffs)** are updated in the browser.

Example Workflow:

```
Old Virtual DOM → New Virtual DOM
        |                    |
         (Diffing Algorithm)
               ↓
   Update Changed Nodes Only
```

Why Virtual DOM is Efficient

- Minimizes direct DOM manipulations.
- Batch updates for performance.
- Ensures smoother rendering for SPAs.

Code Example

```
function Counter() {
  const [count, setCount] = React.useState(0);
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Only `<h1>` re-renders on each click, not the whole page — thanks to Virtual DOM diffing.

Summary Table

Concept	Description	Example	Reference
React	JS library for UIs	`ReactDOM.render()`	Ch 1 p. 3–5
SPA	Single-page dynamic site	React Router	Ch 8 p. 194–195
Vite Setup	Modern React bootstrapping	`npm create vite@latest`	Ch 6 p. 119
Import/Export	ES6 module system	`import {useState}`	Ch 6 p. 124
Virtual DOM	Efficient UI diffing	React updates changed nodes only	Ch 3 p. 52–53